

LẬP TRÌNH CHO TRÍ TUỆ NHÂN TẠO

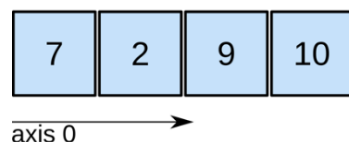
Giảng viên: Nguyễn Ngọc Giang

Numpy

1. Numpy là gì?

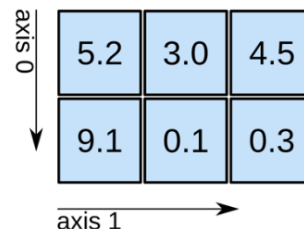
- Numpy (Numerical Python) là thư viện xử lý mảng số học mạnh mẽ trong Python. Được tạo ra từ năm 2005 bởi Travis Oliphant
- Hỗ trợ mảng đa chiều, các phép toán vector hóa hiệu suất cao.
- Được sử dụng rộng rãi trong khoa học dữ liệu, học máy và trí tuệ nhân tạo.

1D array



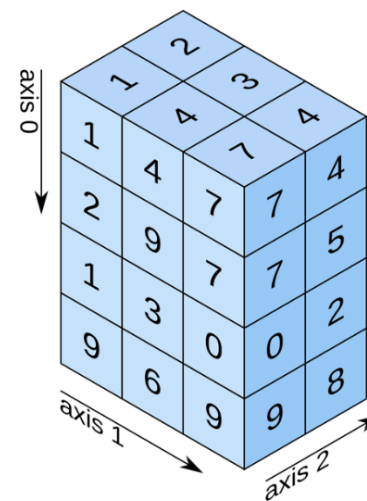
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

1. Numpy là gì?

- Numpy array tốt hơn Python list:
 - Hiệu suất cao hơn: Sử dụng mảng đồng nhất, tối ưu hóa bộ nhớ.
 - Hỗ trợ nhiều phép toán nhanh chóng: Có sẵn các hàm toán học và thống kê.
 - Dễ dàng thao tác với dữ liệu số lớn.
- Numpy một phần được viết bằng Python, còn lại những phần yêu cầu tính toán nhanh thì viết bằng C/C++
- Mã nguồn: <https://github.com/numpy/numpy>

2. Kiểu dữ liệu

STT	Dtype	Meaning
1	bool_	Boolean (True or False) stored as a byte
2	int_	Default integer type (same as C long; normally either int64 or int32)
3	intc	Identical to C int (normally int32 or int64)
4	intp	Integer used for indexing (same as C ssize_t; normally either int32 or int64)
5	int8	Byte (-128 to 127)
6	int16	Integer (-32768 to 32767)
7	int32	Integer (-2147483648 to 2147483647)
8	int64	Integer (-9223372036854775808 to 9223372036854775807)
9	uint8	Unsigned integer (0 to 255)
10	uint16	Unsigned integer (0 to 65535)
11	uint32	Unsigned integer (0 to 4294967295)
12	uint64	Unsigned integer (0 to 18446744073709551615)
13	float_	Shorthand for float64
14	float16	Half precision float: sign bit, 5 bits exponent, 10 bits mantissa
15	float32	Single precision float: sign bit, 8 bits exponent, 23 bits mantissa
16	float64	Double precision float: sign bit, 11 bits exponent, 52 bits mantissa
17	complex_	Shorthand for complex128
18	complex64	Complex number, represented by two 32-bit floats (real and imaginary components)
19	complex128	Complex number, represented by two 64-bit floats (real and imaginary components)

3. Cài đặt và khai báo

Người dùng Mac và Linux có thể cài đặt NumPy thông qua lệnh pip:

```
pip install numpy
```

```
1 | import numpy as np
```

4. Làm việc với mảng (array)

Khởi tạo mảng một chiều

```
#Khởi tạo mảng một chiều với kiểu dữ liệu các phần tử là Integer  
arr = np.array([1,3,4,5,6], dtype = int)  
  
#Khởi tạo mảng một chiều với kiểu dữ liệu mặc định  
arr = np.array([1,3,4,5,6])
```

Khởi tạo mảng hai chiều

```
arr1 = np.array([(4,5,6), (1,2,3)], dtype = int)
```

Khởi tạo mảng ba chiều

```
arr2 = np.array([(2,4,0,6), (4,7,5,6)],  
                [(0,3,2,1), (9,4,5,6)],  
                [(5,8,6,4), (1,4,6,8)]), dtype = int)
```

4.1 Khởi tạo array

Scalar

1

Vector

$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ or $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$

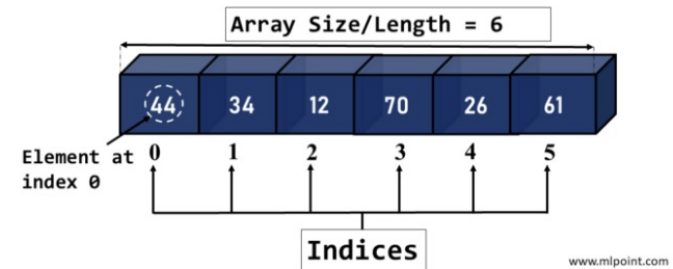
Matrix

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$

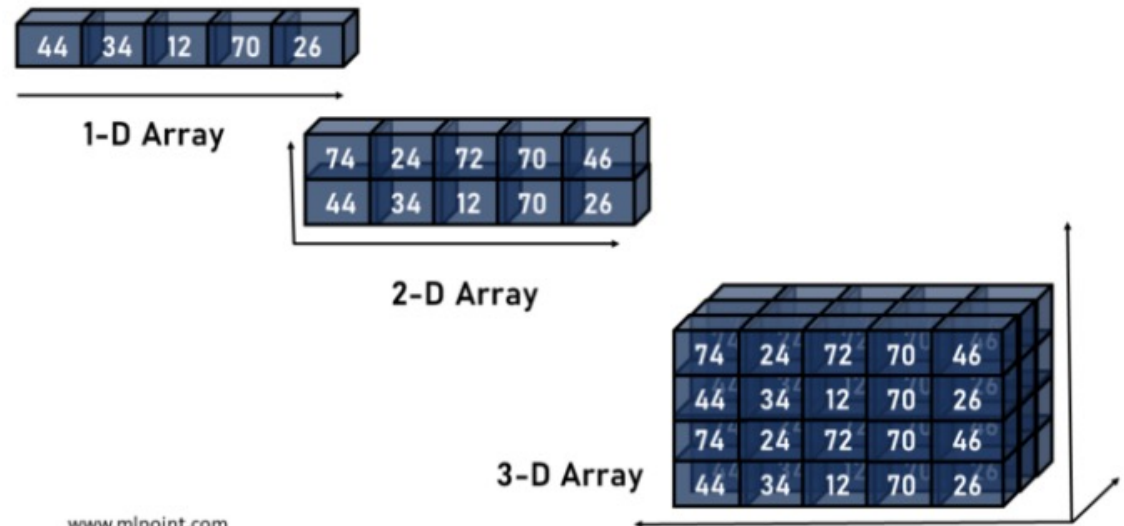
Tensor

$\begin{bmatrix} [1 & 2] & [3 & 4] \\ [5 & 6] & [7 & 8] \\ [9 & 0] & [1 & 2] \end{bmatrix}$

1-D Array with 6 Elements



Arrays of Different Dimensions

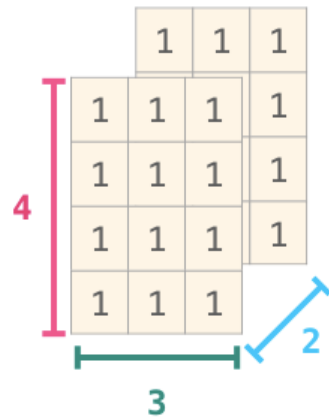


4.1 Khởi tạo array

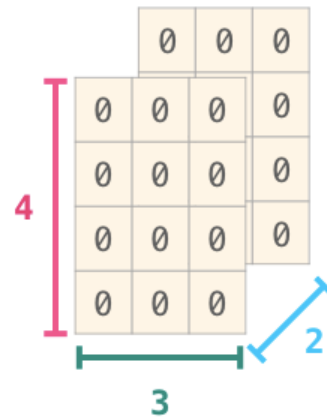
- Dùng các hàm như: `np.empty([3, 4], dtype=int)`
 - Hoặc các hàm dưới đây
 - `np.ones`, `np.zeros`
 - `np.arange`
 - `np.concatenate`
 - `np.astype`
 - `np.zeros_like`,
`np.ones_like`
 - `np.random.random`
-
- `np.zeros((3,4), dtype = int)`: Tạo mảng hai chiều các phần tử 0 với kích thước 3x4.
 - `np.ones((2,3,4), dtype = int)`: Tạo mảng 3 chiều các phần tử 1 với kích thước 2x3x4.
 - `np.arange(1,7,2)`: Tạo mảng với các phần tử từ 1 - 6 với bước nhảy là 2.
 - `np.full((2,3),5)`: Tạo mảng 2 chiều các phần tử 5 với kích thước 2x3.
 - `np.eye(4, dtype=int)`: Tạo ma trận đơn vị với kích thước là 4x4.
 - `np.random.random((2,3))`: Tạo ma trận các phần tử ngẫu nhiên với kích thước 2x3.

4.1 Khởi tạo array

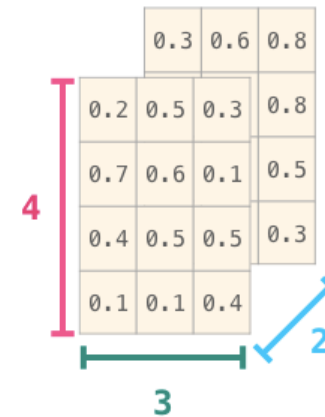
`np.ones((4,3,2))`



`np.zeros((4,3,2))`



`np.random.random((4,3,2))`



```
>>> np.arange(1334,1338)
array([1334, 1335, 1336, 1337])
```

```
>>> A = np.ones((4,1))
>>> B = np.zeros((4,2))
>>> np.concatenate([A,B], axis=1)
array([[ 1.,  0.,  0.],
       [ 1.,  0.,  0.],
       [ 1.,  0.,  0.],
       [ 1.,  0.,  0.]])
```

4.1 Khởi tạo array

```
>>> A
array([[ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5]], dtype=float32)
>>> print(A.astype(np.uint16))
[[4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]]
```

```
>>> a = np.ones((2,2,3))
>>> b = np.zeros_like(a)
>>> print(b.shape)
```

```
>>> np.random.random((10,3))
array([[ 0.61481644,  0.55453657,  0.04320502],
       [ 0.08973085,  0.25959573,  0.27566721],
       [ 0.84375899,  0.2949532 ,  0.29712833],
       [ 0.44564992,  0.37728361,  0.29471536],
       [ 0.71256698,  0.53193976,  0.63061914],
       [ 0.03738061,  0.96497761,  0.01481647],
       [ 0.09924332,  0.73128868,  0.22521644],
       [ 0.94249399,  0.72355378,  0.94034095],
       [ 0.35742243,  0.91085299,  0.15669063],
       [ 0.54259617,  0.85891392,  0.77224443]])
```

4.2 Các thuộc tính trong đối tượng mảng

- **dtype**: Kiểu dữ liệu của phần tử trong mảng.
- **shape**: Kích thước của mảng.
- **size**: Số phần tử trong mảng.
- **ndim**: Số chiều của mảng.

```
1 print("Kiểu dữ liệu của phần tử trong mảng:", arr2.dtype)
2 print("Kích thước của mảng:", arr2.shape)
3 print("Số phần tử trong mảng:", arr2.size)
4 print("Số chiều của mảng:", arr2.ndim)
```

Output:

Kiểu dữ liệu của phần tử trong mảng: int32

Kích thước của mảng: (3, 2, 4)

Số phần tử trong mảng: 24

Số chiều của mảng: 3

4.2 Các thuộc tính trong đối tượng mảng

data

1	2
3	4
5	6

data.T

1	3	5
2	4	6

data

1
2
3
4
5
6

data.reshape(2,3)

1	2	3
4	5	6

Diagram illustrating the reshape operation: A vertical array of 6 elements is reshaped into a 2x3 matrix. The dimensions are indicated by a red vertical bracket labeled '2' and a purple horizontal bracket labeled '3'.

data.reshape(3,2)

1	2
3	4
5	6

Diagram illustrating the reshape operation: A vertical array of 6 elements is reshaped into a 3x2 matrix. The dimensions are indicated by a red vertical bracket labeled '3' and a purple horizontal bracket labeled '2'.

4.3 Chỉ mục và lát cắt trong array

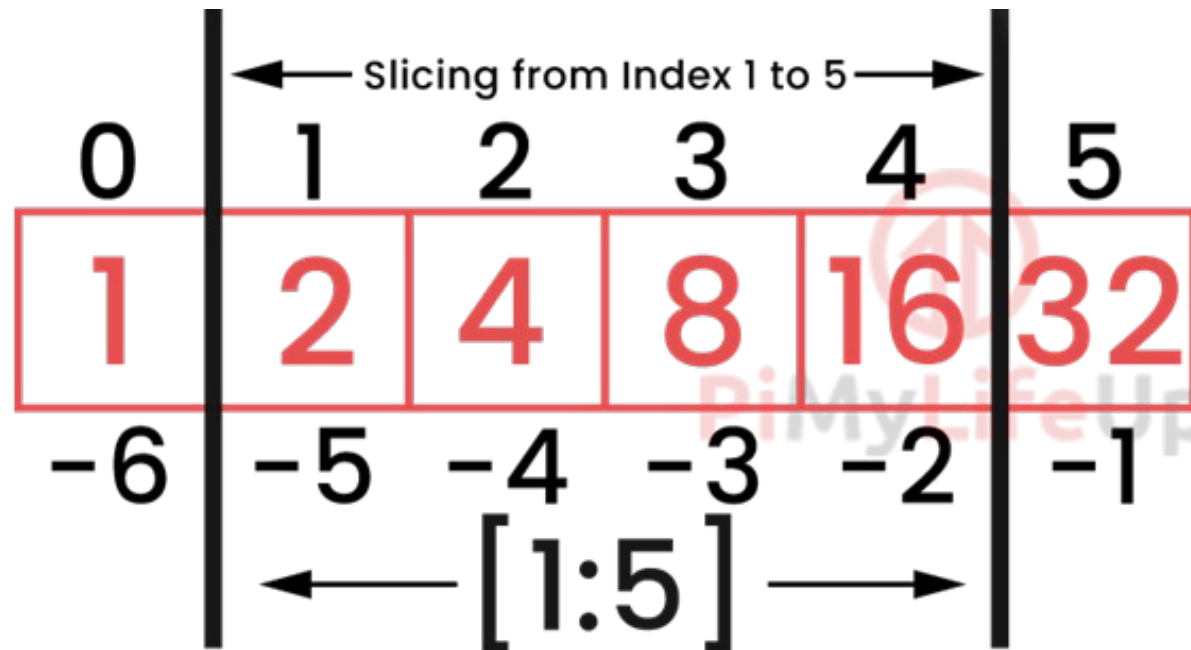
Indexing: Mỗi thành phần trong mảng 1 chiều tương ứng với một chỉ số. Chỉ số trong NumPy, cũng giống như chỉ số trong python, bắt đầu bằng 0. Nếu mảng 1 chiều có n phần tử thì các chỉ số chạy từ 0 đến $n-1$

Slicing: Giống như list trong python, numpy array cũng có thể được cắt (slicing) theo 1 chiều, hoặc nhiều chiều.

- `arr[i]`: Truy cập tới phần tử thứ i của mảng 1 chiều.
- `arr1[i,j]`: Truy cập tới phần tử hàng i , cột j của mảng 2 chiều.
- `arr2[n,i,j]`: Truy cập tới phần tử chiều n , hàng i , cột j của mảng 3 chiều.
- `arr[a:b]`: Truy cập tới các phần tử từ a đến $b-1$ trong mảng 1 chiều.
- `arr1[:,i]`: Truy cập tới phần tử từ cột 0 đến cột $i-1$, của tất cả các hàng trong mảng 2 chiều.

4.3 Chỉ mục và lát cắt trong array

	1	3	5	7	9
index	0	1	2	3	4
negative index	-5	-4	-3	-2	-1



4.3 Chỉ mục và lát cắt trong array

	data	data[0]	data[1]	data[0:2]	data[1:]
0	1	1		1	
1	2		2	2	2
2	3				3

	data		data[0,1]		data[1:3]		data[0:2,0]	
	0	1	0	1	0	1	0	1
0	1	2	1	2	1	2	1	2
1	3	4	3	4	3	4	3	4
2	5	6	5	6	5	6	5	6

4.3 Chỉ mục và lát cắt trong array

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Row one, columns two to four

```
>>> arr[1, 2:4]  
array([7, 8])
```

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

All rows in column one

```
>>> arr[:, 1]  
array([2, 6, 10, 14])
```

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

All rows after row two,
all columns after column two

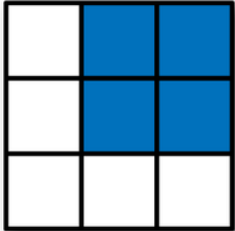
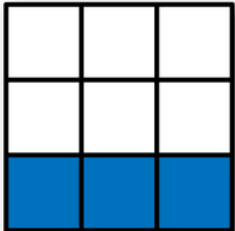
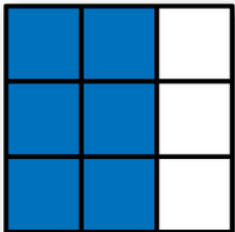
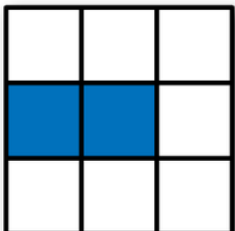
```
>>> arr[2:, 2:]  
array([[11, 12],  
       [15, 16]])
```

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

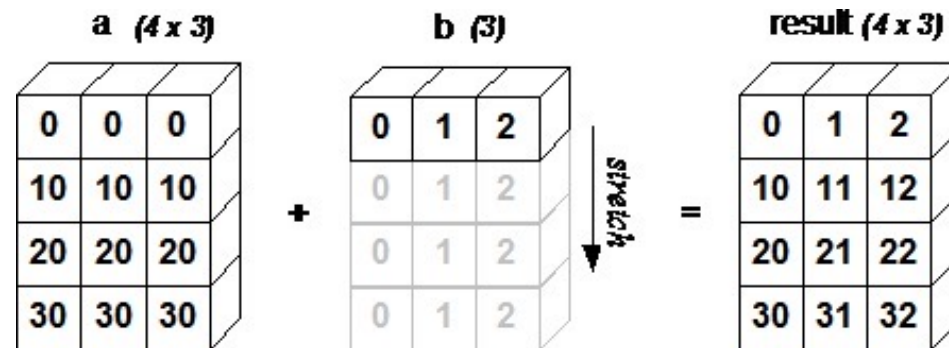
Every other row after row one,
every other column

```
>>> arr[1::2, ::2]  
array([[5, 7],  
       [13, 15]])
```

4.3 Chỉ mục và lát cắt trong array

	Expression	Shape
	<code>arr[:2, 1:]</code>	<code>(2, 2)</code>
	<code>arr[2]</code> <code>arr[2, :]</code> <code>arr[2:, :]</code>	<code>(3,)</code> <code>(3,)</code> <code>(1, 3)</code>
	<code>arr[:, :2]</code>	<code>(3, 2)</code>
	<code>arr[1, :2]</code> <code>arr[1:2, :2]</code>	<code>(2,)</code> <code>(1, 2)</code>

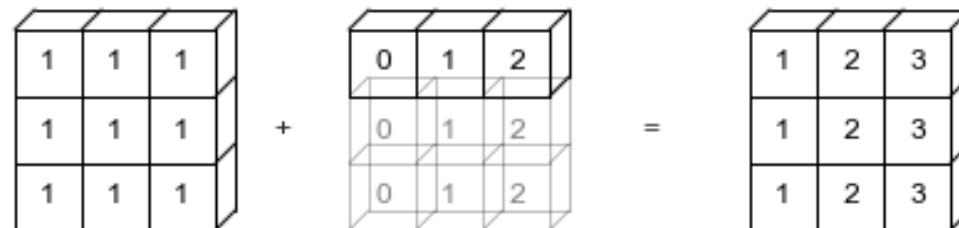
4.4 Broadcasting



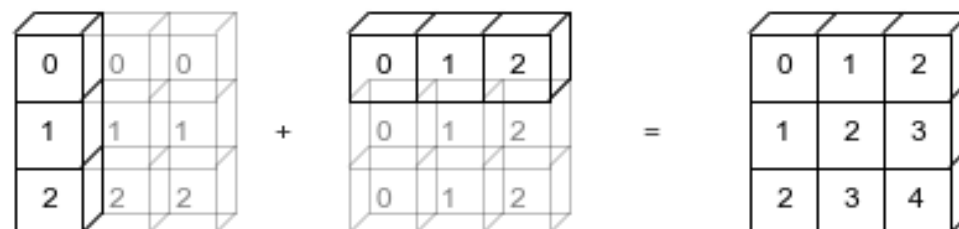
`np.arange(3)+5`



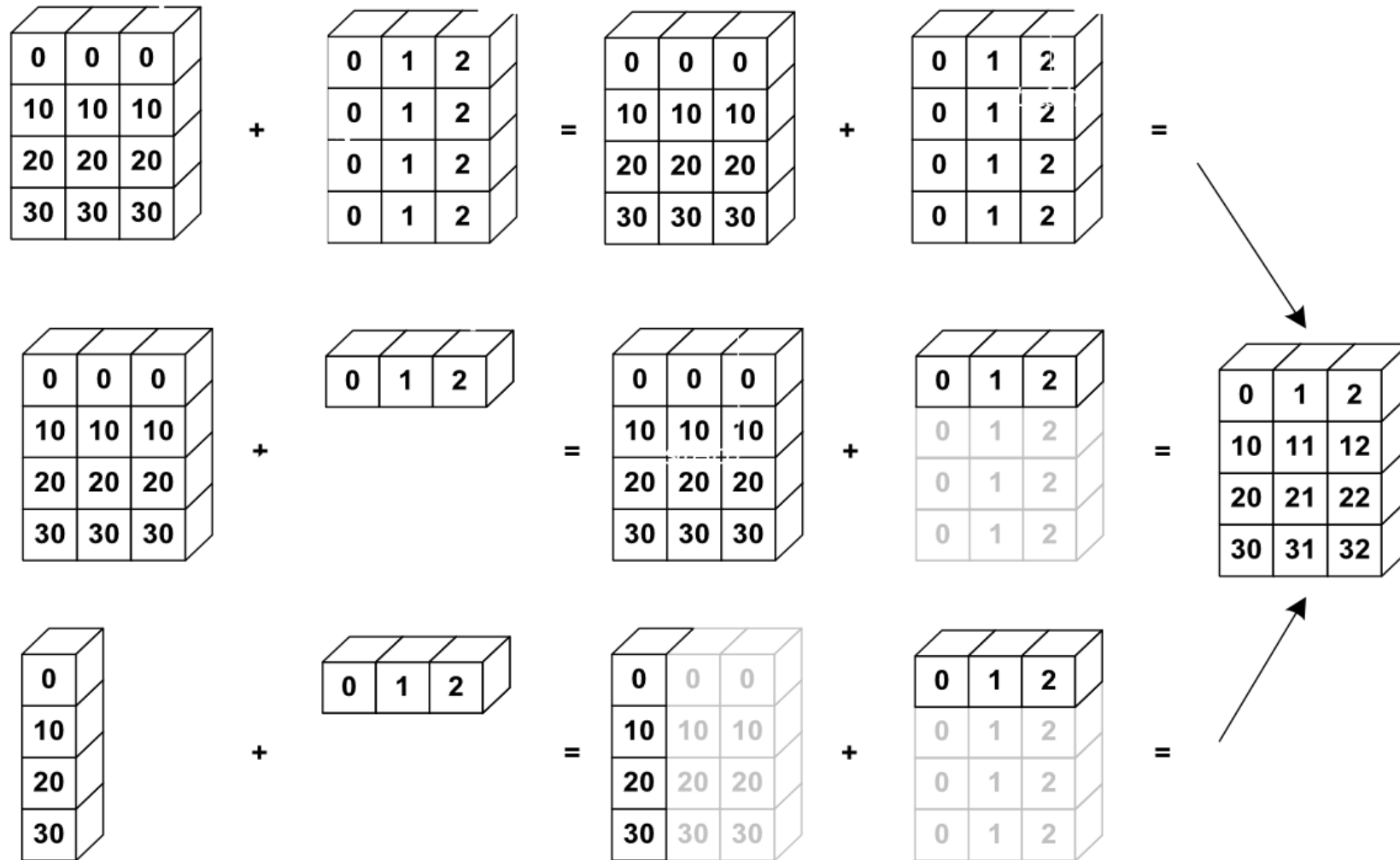
`np.ones((3,3))+np.arange(3)`



`np.arange(3).reshape((3,1))+np.arange(3)`



4.4 Broadcasting



4.5 Thao túng mảng

Changing Shape

Sr.No.	Shape & Description
1	<code>reshape</code>  Gives a new shape to an array without changing its data
2	<code>flat</code>  A 1-D iterator over the array
3	<code>flatten</code>  Returns a copy of the array collapsed into one dimension
4	<code>ravel</code>  Returns a contiguous flattened array

4.5 Thao túng mảng

Transpose Operations

Sr.No.	Operation & Description
1	<code>transpose</code>  Permutes the dimensions of an array
2	<code>ndarray.T</code>  Same as <code>self.transpose()</code>
3	<code>rollaxis</code>  Rolls the specified axis backwards
4	<code>swapaxes</code>  Interchanges the two axes of an array

4.5 Thao túng mảng

Changing Dimensions

Sr.No.	Dimension & Description
1	<code>broadcast</code> ↗ Produces an object that mimics broadcasting
2	<code>broadcast_to</code> ↗ Broadcasts an array to a new shape
3	<code>expand_dims</code> ↗ Expands the shape of an array
4	<code>squeeze</code> ↗ Removes single-dimensional entries from the shape of an array

4.5 Thao túng mảng

Joining Arrays

Sr.No.	Array & Description
1	<code>concatenate</code>  Joins a sequence of arrays along an existing axis
2	<code>stack</code>  Joins a sequence of arrays along a new axis
3	<code>hstack</code>  Stacks arrays in sequence horizontally (column wise)
4	<code>vstack</code>  Stacks arrays in sequence vertically (row wise)

4.5 Thao túng mảng

Adding / Removing Elements

Sr.No.	Element & Description
1	resize ↗ Returns a new array with the specified shape
2	append ↗ Appends the values to the end of an array
3	insert ↗ Inserts the values along the given axis before the given indices
4	delete ↗ Returns a new array with sub-arrays along an axis deleted
5	unique ↗ Finds the unique elements of an array

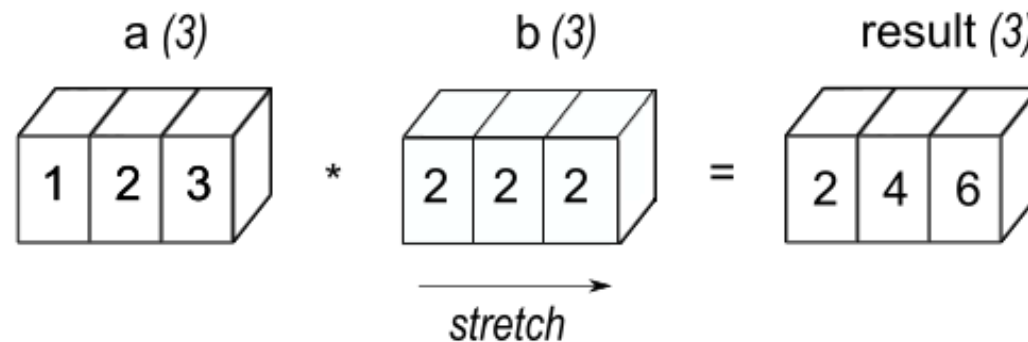
4.5 Thao túng mảng

Splitting Arrays

Sr.No.	Array & Description
1	<code>split</code>  Splits an array into multiple sub-arrays
2	<code>hsplit</code>  Splits an array into multiple sub-arrays horizontally (column-wise)
3	<code>vsplit</code>  Splits an array into multiple sub-arrays vertically (row-wise)

4.6 Các phép toán số học

- Cộng: $\text{arr1} + \text{arr2}$
- Trừ: $\text{arr1} - \text{arr2}$
- Nhân: $\text{arr1} * \text{arr2}$
- Chia: $\text{arr1} / \text{arr2}$
- Bình phương: $\text{arr} ** 2$



4.7 Các hàm thống kê

data

1
2
3

$$.max() = 3$$

data

1
2
3

$$.min() = 1$$

data

1
2
3

$$.sum() = 6$$

data

1	2
3	4
5	6

$$.max() = 6$$

data

1	2
3	4
5	6

$$.min() = 1$$

data

1	2
3	4
5	6

$$.sum() = 21$$

data

1	2
5	3
4	6

$$.max(axis=0) = \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 5 & 3 \\ \hline 4 & 6 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 5 & 6 \\ \hline \end{array}$$

data

1	2
5	3
4	6

$$.max(axis=1) = \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 5 & 3 \\ \hline 4 & 6 \\ \hline \end{array} = \begin{array}{|c|} \hline 2 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}$$

4.7 Các hàm thống kê

NumPy Function	Description
<code>np.mean</code>	The average of all values in the array.
<code>np.std</code>	Standard deviation.
<code>np.var</code>	Variance.
<code>np.sum</code>	Sum of all elements.
<code>np.prod</code>	Product of all elements.
<code>np.cumsum</code>	Cumulative sum of all elements.
<code>np.cumprod</code>	Cumulative product of all elements.
<code>np.min</code> , <code>np.max</code>	The minimum/maximum value in an array.
<code>np.argmin</code> , <code>np.argmax</code>	The index of the minimum/maximum value in an array.
<code>np.all</code>	Returns True if all elements in the argument array are nonzero.
<code>np.any</code>	Returns True if any of the elements in the argument array is nonzero.

4.8 Các hàm toán học

NumPy Function	Description
<code>np.cos</code> , <code>np.sin</code> , <code>np.tan</code>	Trigonometric functions.
<code>np.arccos</code> , <code>np.arcsin</code> , <code>np.arctan</code>	Inverse trigonometric functions.
<code>np.cosh</code> , <code>np.sinh</code> , <code>np.tanh</code>	Hyperbolic trigonometric functions.
<code>np.arccosh</code> , <code>np.arcsinh</code> , <code>np.arctanh</code>	Inverse hyperbolic trigonometric functions.
<code>np.sqrt</code>	Square root.
<code>np.exp</code>	Exponential.
<code>np.log</code> , <code>np.log2</code> , <code>np.log10</code>	Logarithms of base e, 2, and 10, respectively.

4.8 Các hàm toán học

NumPy Function	Description
<code>np.add</code> , <code>np.subtract</code> , <code>np.multiply</code> , <code>np.divide</code>	Addition, subtraction, multiplication, and division of two NumPy arrays.
<code>np.power</code>	Raises first input argument to the power of the second input argument (applied elementwise).
<code>np.remainder</code>	The remainder of division.
<code>np.reciprocal</code>	The reciprocal (inverse) of each element.
<code>np.real</code> , <code>np.imag</code> , <code>np.conj</code>	The real part, imaginary part, and the complex conjugate of the elements in the input arrays.
<code>np.sign</code> , <code>np.abs</code>	The sign and the absolute value.
<code>np.floor</code> , <code>np.ceil</code> , <code>np rint</code>	Convert to integer values.
<code>np.round</code>	Rounds to a given number of decimals.

4.9 Sort, Search & Counting

kind	speed	worst case	work space	stable
'quicksort'	1	$O(n^2)$	0	no
'mergesort'	2	$O(n \log(n))$	$\sim n/2$	yes
'heapsort'	3	$O(n \log(n))$	0	no

`Np.sort()`

`Np.argsort()`

`Np.max()`

`Np.min()`

`Np.argmax()`

`Np.argmin()`

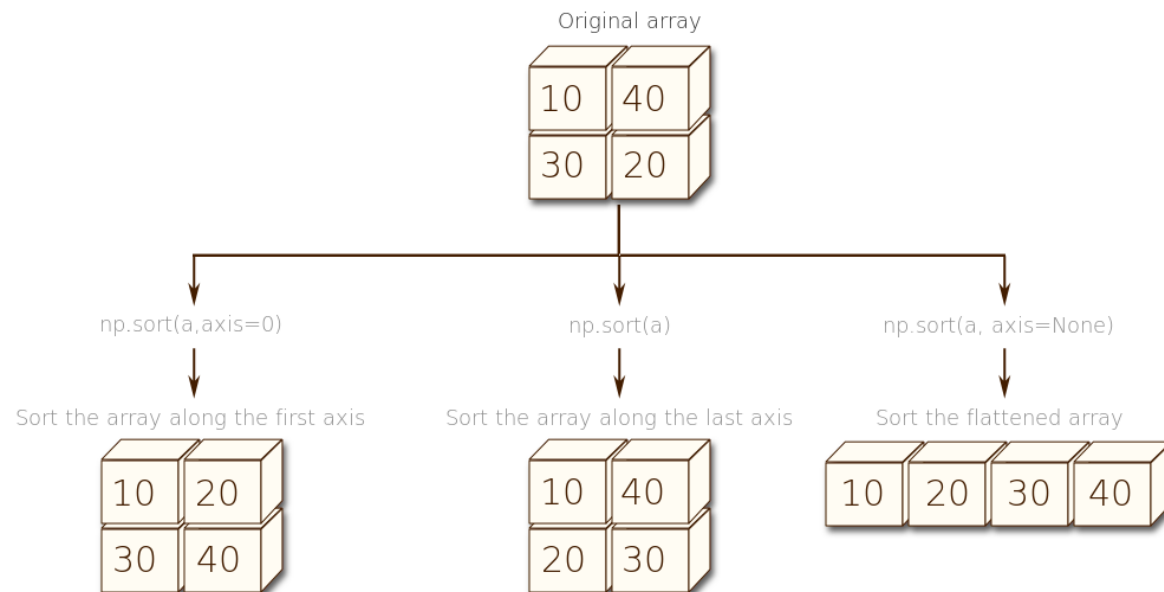
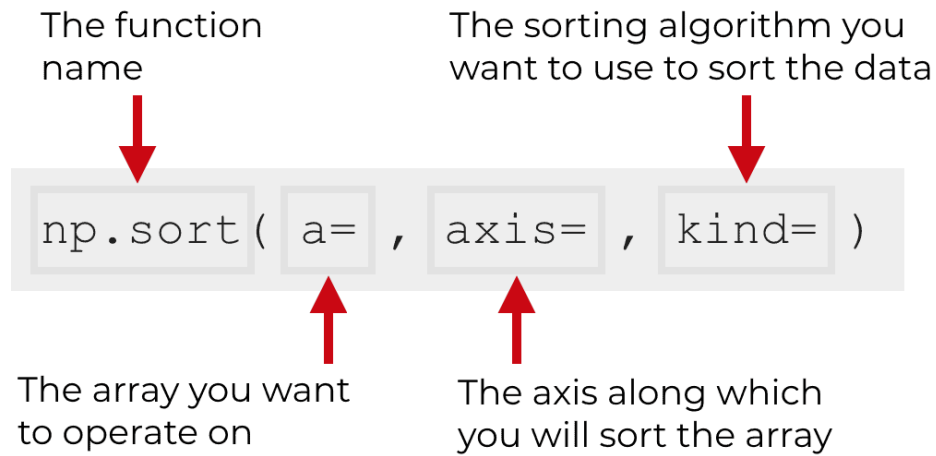
`Np.nonzero()`

`Np.where()`

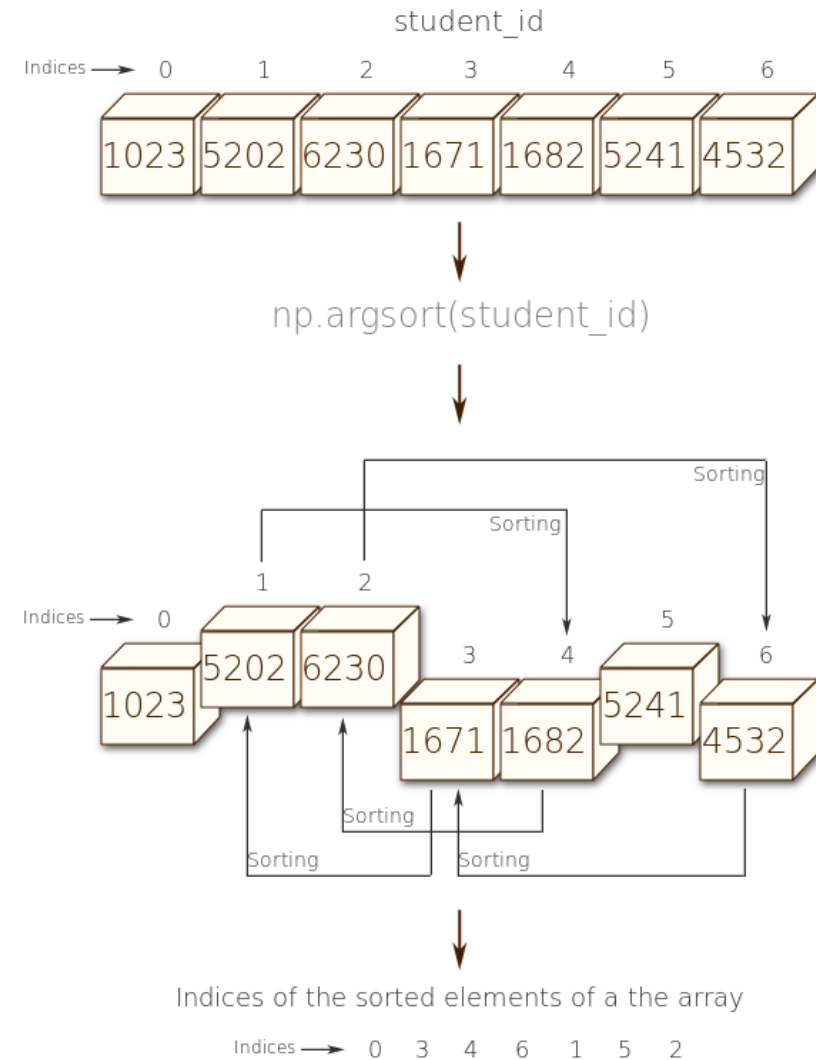
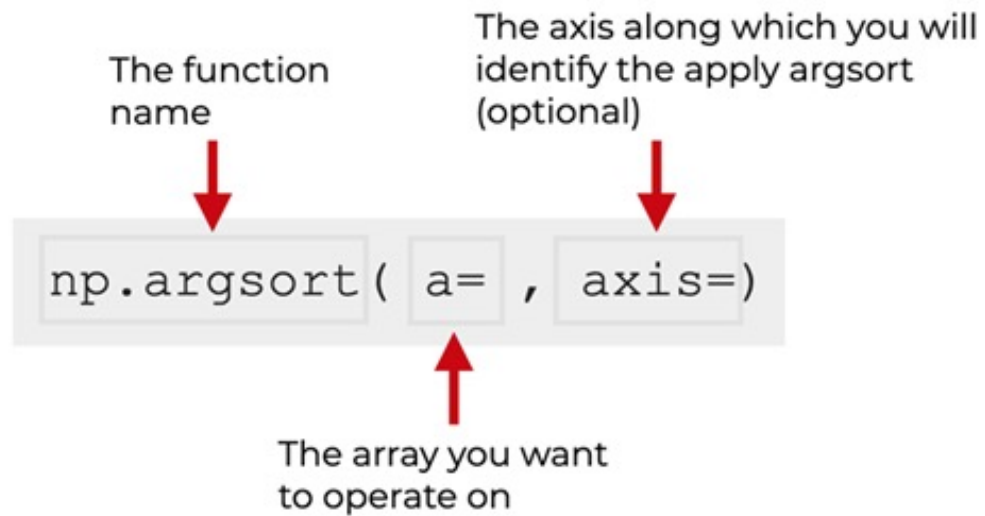
`Np.extract()`

`Np.searchsorted()`

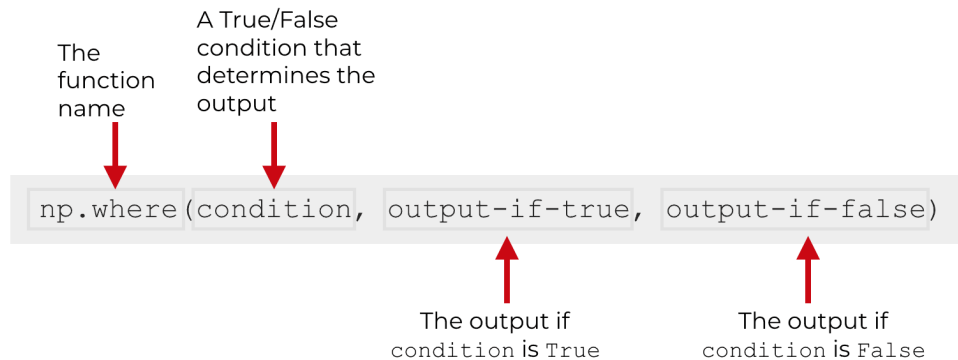
Np.sort



Np.argsort



Np.where



```
In [2]: import numpy as np
```

```
In [15]: np.where([True, False, False], [1, 2, 4], [7, 8, 9])
```

```
Out[15]: array([1, 8, 9])
```

```
In [2]: import numpy as np
```

```
In [3]: arr = np.array([11, 12, 13, 14])
```

```
In [4]: high_values = ['High', 'High', 'High', 'High']  
low_values = ['Low', 'Low', 'Low', 'Low']
```

```
In [5]: arr > 12
```

```
Out[5]: array([False, False,  True,  True])
```

```
In [6]: np.where(arr > 12, ['High', 'High', 'High', 'High'], ['Low', 'Low', 'Low', 'Low'])
```

```
Out[6]: array(['Low', 'Low', 'High', 'High'], dtype='<U4')
```

Np.where

```
In [2]: import numpy as np
```

```
In [10]: arr = np.array([11, 12, 14, 15, 16, 17])
```

```
In [14]: np.where( (arr > 12) & (arr < 16))
```

```
Out[14]: (array([2, 3], dtype=int64),)
```

4.10 Đại số tuyến tính với mảng

Sr.No.	Function & Description
1	<code>dot</code>  Dot product of the two arrays
2	<code>vdot</code>  Dot product of the two vectors
3	<code>inner</code>  Inner product of the two arrays
4	<code>matmul</code>  Matrix product of the two arrays
5	<code>determinant</code>  Computes the determinant of the array
6	<code>solve</code>  Solves the linear matrix equation
7	<code>inv</code>  Finds the multiplicative inverse of the matrix

np.dot

$$s = \mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

$$\mathbf{a} = \begin{bmatrix} 3 & 4 & 5 \end{bmatrix}$$

$$\mathbf{b} = \begin{bmatrix} 7 & 8 & 9 \end{bmatrix}$$

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= 3 \times 7 + 4 \times 8 + 5 \times 9 \\ &= 21 + 32 + 45 \\ &= 98 \end{aligned}$$

$$r = 2$$

$$\mathbf{b} = \begin{bmatrix} 7 & 8 & 9 \end{bmatrix}$$

$$\begin{aligned} r\mathbf{b} &= \begin{bmatrix} 2 \times 7 & 2 \times 8 & 2 \times 9 \end{bmatrix} \\ &= \begin{bmatrix} 14 & 16 & 18 \end{bmatrix} \end{aligned}$$

np.dot

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}$$

$$\mathbf{A} = \begin{bmatrix} 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} 10 & 11 \\ 12 & 13 \\ 14 & 15 \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} 3 \times 10 + 4 \times 12 + 5 \times 14 & 3 \times 11 + 4 \times 13 + 5 \times 15 \\ 6 \times 10 + 7 \times 12 + 8 \times 14 & 6 \times 11 + 7 \times 13 + 8 \times 15 \end{bmatrix}$$

The function name

The second input array

The first input array

```
np.dot(a, b)
```

$$\mathbf{AB} = \begin{bmatrix} 148 & 160 \\ 256 & 277 \end{bmatrix}$$

Nhân Ma trận

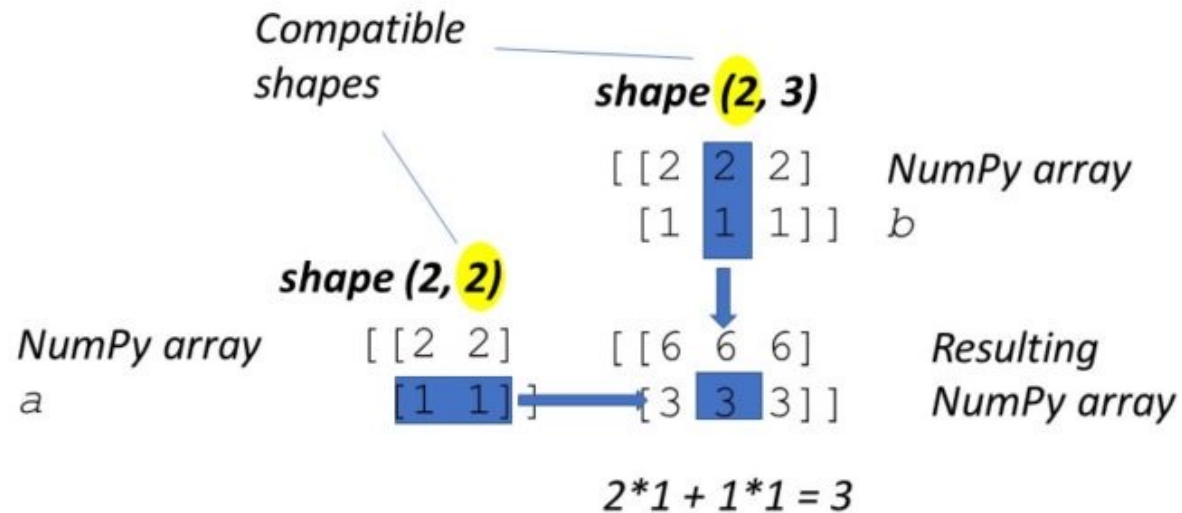
finxter

`a@b`

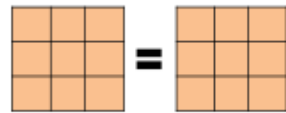
`np.dot(a, b)`

`np.matmul(a, b)`

*Equivalent Semantics:
Matrix Multiplication*



Nhân Ma trận



Equality of
Matrices

Addition of
Matrices

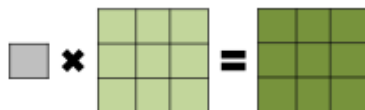


Subtraction of
Matrices

Multiplication of Matrices

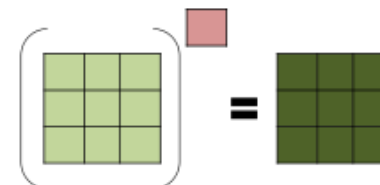
$$\begin{pmatrix} 1 & 1 \end{pmatrix} + \begin{pmatrix} 2 & 2 \end{pmatrix} + \begin{pmatrix} 3 & 3 \end{pmatrix} = \begin{pmatrix} 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \end{pmatrix} \times \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 3 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$



Matrix
Multiplied by
a Scalar

Power of a
Matrix



codeformech.com

Định thức

$$|A| = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc.$$

Similarly, for a 3×3 matrix A , its determinant is:

$$\begin{aligned} |A| = \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} &= a \begin{vmatrix} \square & \square & \square \\ \square & e & f \\ \square & h & i \end{vmatrix} - b \begin{vmatrix} \square & \square & \square \\ d & \square & f \\ g & \square & i \end{vmatrix} + c \begin{vmatrix} \square & \square & \square \\ d & e & \square \\ g & h & \square \end{vmatrix} = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix} \\ &= aei + bfg + cdh - ceg - bdi - afh. \end{aligned}$$

Định thức

Matrix determinants

9. Compute the determinant for the coefficient matrix in question 2. What do you observe?

10. For the matrix

$$A = \begin{bmatrix} 2 & 3 & -1 \\ 1 & 1 & 3 \\ 1 & 2 & -1 \end{bmatrix}$$

compute the determinant twice, first by expanding about the top row and second by expanding about the second column.

11. Given

$$A = \begin{bmatrix} 1 & 1 \\ 1 & -4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & -1 \\ 3 & 1 \end{bmatrix}$$

compute $\det(A)$, $\det(B)$ and $\det(AB)$. Verify that $\det(AB) = \det(A)\det(B)$.

12. Compute the following determinants using expansions about any suitable row or column.

(a) $\det\left(\begin{bmatrix} 1 & 2 & 3 \\ 3 & 2 & 2 \\ 0 & 9 & 8 \end{bmatrix}\right)$

(b) $\det\left(\begin{bmatrix} 4 & 3 & 2 \\ 1 & 7 & 8 \\ 3 & 9 & 3 \end{bmatrix}\right)$

(c) $\det\left(\begin{bmatrix} 1 & 2 & 3 & 2 \\ 1 & 3 & 2 & 3 \\ 4 & 0 & 5 & 0 \\ 1 & 2 & 1 & 2 \end{bmatrix}\right)$

(d) $\det\left(\begin{bmatrix} 1 & 5 & 1 & 3 \\ 2 & 1 & 7 & 5 \\ 1 & 2 & 1 & 0 \\ 3 & 1 & 0 & 1 \end{bmatrix}\right)$

13. Recompute the determinants in the previous question this time using row operations (that is, Gaussian elimination).

4.11 Numpy Random

From numpy import random

1. Randint()
2. Rand()
3. Choice()
4. Shuffle()
5. Permutation()

6. Normal()
7. Binormal()
8. Poisson()
9. Uniform()
10. Logistic()
11. Multinomial()
12. Exponential()
13. Chisquare()
14. ...

4.12 I/O Numpy

Chúng ta có thể lưu numpy dưới nhiều định dạng khác nhau như file npy và txt.

Save mảng A theo hàm `np.save()` và `np.savetxt()` có cú pháp:

- `np.save(file, arr)`: Output là định dạng npy. Save được mọi tensor với số chiều tùy ý.
- `np.savetxt(file, arr)`: Output là định dạng txt. Cách này ít phổ biến hơn vì ta chỉ save được mảng 1D và 2D.

Trong các công thức trên thì `file` là đường link tới vị trí save và `arr` là mảng cần save.

```
# Định dạng npy
np.save('numpy_save/A', A)
# Định dạng txt
np.savetxt('numpy_save/A.txt', A)
```

4.12 I/O Numpy

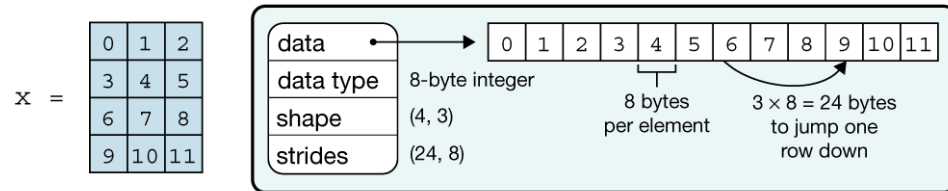
Sau khi save numpy, ta có thể load chúng lại theo hàm `np.load()` và `np.loadtxt()`.

```
A = np.load("numpy_save/A.npy")  
print(A)
```

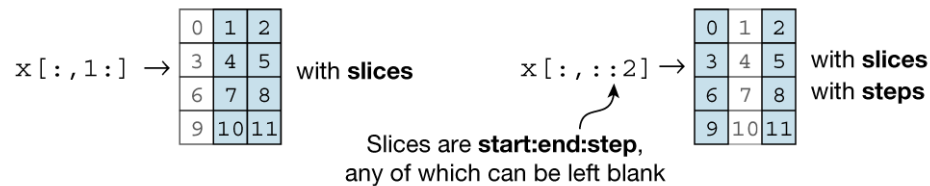
```
np.loadtxt("numpy_save/A.txt")  
print(A)
```

4.13 Tổng kết numpy

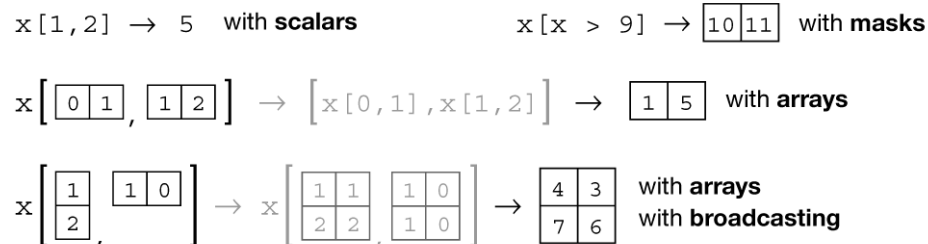
a Data structure



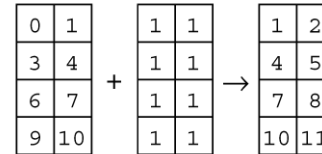
b Indexing (view)



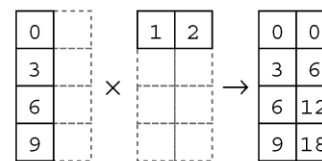
c Indexing (copy)



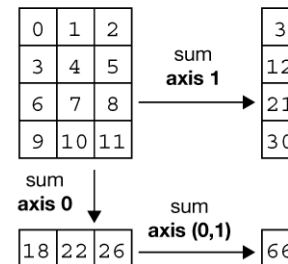
d Vectorization



e Broadcasting



f Reduction



g Example

```
In [1]: import numpy as np
In [2]: x = np.arange(12)
In [3]: x = x.reshape(4, 3)

In [4]: x
Out[4]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])

In [5]: np.mean(x, axis=0)
Out[5]: array([4.5, 5.5, 6.5])

In [6]: x = x - np.mean(x, axis=0)

In [7]: x
Out[7]:
array([[ -4.5,  -4.5,  -4.5],
       [ -1.5,  -1.5,  -1.5],
       [  1.5,   1.5,   1.5],
       [  4.5,   4.5,   4.5]])
```

5. Ứng dụng thực tế của Numpy

- Xử lý dữ liệu lớn nhanh chóng.
- Tạo và xử lý dữ liệu đầu vào cho Machine Learning.
- Dùng trong các thuật toán tối ưu hóa, xử lý ảnh.

6. Thực hành và bài tập

- Link thực hành và bài tập trên Colab:
- https://colab.research.google.com/drive/1LGmYfRj6uvFOa11afllkZWlc_SDbN8O6?usp=sharing